

GentleStar: Safe Navigation in Duckietown

Yahia Taha¹, Ahmad Kaihan Saba¹, and Đorđe Vidaković¹

¹Technical University of Munich (TUM)

{yahia.taha, kaihan.saba, djordje.vidakovic}@tum.de

Abstract – We present the design and implementation of an autonomous navigation stack for a Duckiebot (“duck3”) that safely drives across the TUM Duckietown lab track from a designated start tile to a designated goal tile. The system follows the lane using classical computer vision (HSV color thresholding and a Proportional–Derivative controller), stops at red stop lines, plans the route across intersections using a weighted A* search over the track’s tile graph, and detects and avoids rubber ducks placed on the road using a shape-based contour classifier. All computation runs off-robot on a laptop, communicating with the Duckiebot over rosbbridge; no custom code is deployed to the robot itself. We additionally report on an attempted reinforcement-learning (PPO) approach trained in simulation, which did not transfer to the real robot due to the sim-to-real gap, and describe why we reverted to the classical pipeline described in this report.

1 Introduction

Duckietown is a low-cost robotics platform used to teach and research autonomous driving on a miniature scale: small differential-drive robots (“Duckiebots”) drive on a tile-based road network painted with lane markings, stop lines, and intersections, sharing the road with rubber duck “pedestrians.” The goal of this project was to make our assigned robot, duck3, safely navigate the TUM lab track from a fixed starting tile to a fixed goal tile, following standard lane markings, stopping correctly at intersections, and avoiding collisions with ducks placed anywhere along the route.

Our system is organized around three cooperating capabilities, each covered in its own section below:

- **Lane Following** (Section 2): continuous vision-based steering that keeps the robot centered in its lane.
- **Intersection Navigation** (Section 3): global route planning over the track’s tile map, and the local maneuvers used to physically execute a turn.
- **Object Detection and Avoidance** (Section 4): recognizing rubber ducks on the road and safely routing around them without leaving the lane-following loop.

All three run inside a single Python process on a laptop connected to duck3 over its rosbbridge WebSocket (port 9001) via the `roslibpy` library. The laptop subscribes to the robot’s compressed camera stream, and publishes velocity commands (`Twist2DStamped`) back to it. No custom code runs on the robot’s own compute – only the standard Duckietown Docker containers (camera driver, wheel interface, rosbbridge) are active on duck3 itself. A small Flask dashboard, served from the same process, shows the live annotated camera feed, lets an operator pick a start tile and goal tile on a rendered map, and exposes manual keyboard control as a fallback.

2 Lane Following

2.1 Perception

Every incoming camera frame is converted from BGR to HSV color space, which separates hue from brightness and is considerably more robust to the lab’s lighting than thresholding in RGB directly. Two color masks are built per frame: **yellow** (the lane’s centre line, and incidentally the ducks), hue 10–40 with wide saturation/value ranges to tolerate glare and shadow; and **white** (the outer lane edge), low saturation (≤ 45) and high value (≥ 150).

Only the bottom 40% of the frame is searched for lane lines – the section of road immediately in front of the robot, where perspective distortion is smallest and the lines are most reliable. Within that strip, the search is further gated horizontally: yellow is only searched for in the left 70% of the image and white only from the right 30%, reflecting the fixed left-to-right layout of a lane as seen by a forward-facing camera (white edge, oncoming lane, yellow centre line, our lane, white edge). Without these gates the detector could occasionally lock onto the wrong-side line, especially on a curve, and steer the robot off the road. Each mask’s horizontal centroid is computed as an image moment (M_{10}/M_{00}), and discarded as noise if fewer than 500 pixels support it.

The lane center for a given frame is taken as the midpoint between the yellow and white detections when both are visible; if only one line is visible, the center is estimated by offsetting a fixed fraction of the image width (25%) from that single line. If neither line is found, the lane is considered “lost” for that frame.

2.2 Control

Steering uses a standard Proportional–Derivative (PD) controller on the horizontal pixel error e between the lane center

and the image center:

$$\omega = K_P \cdot e + K_D \cdot (e - e_{prev}) \quad (1)$$

with $K_P = 4.0$ and $K_D = 2.0$. The proportional term reacts to how far off-center the robot currently is; the derivative term reacts to how quickly that error is changing, damping oscillation rather than letting the robot overshoot side to side. Forward speed is held at a base value of 0.10 (never dropping below a floor of 0.05), and is reduced in proportion to how hard the robot is steering so that sharp corrections slow the robot down, similar to how a car naturally decelerates through a tight turn. Turn rate is clamped to a maximum magnitude of 6.0 rad/s.

If the lane is briefly lost, the robot does not stop immediately: it continues turning at a decayed fraction of its last steering command (0.85 per frame), fading toward straight. Only after 15 consecutive lost frames does the robot give up and stop, rather than risk driving blind indefinitely.

3 Intersection Navigation

3.1 Track Representation and Path Planning

The TUM track is modeled as a grid of tiles loaded from a YAML map (`tum_map.yaml`, 7×6), each labeled with a tile kind (straight, curve-left, curve-right, a 3-way or 4-way intersection, or grass for non-drivable area) and an orientation. Each kind defines a fixed set of compass directions (N/E/S/W) it can be entered or exited from; two adjacent tiles are considered connected only if each one’s opening faces the other. This produces a graph over which we run a weighted A* search from the robot’s start tile and heading to the chosen goal tile.

Every ordinary tile costs 1.0 to traverse. Any tile that is a real stop-line intersection additionally costs a large fixed penalty (1000.0), because crossing one always requires a full stop-and-wait plus a turn maneuver – far more expensive in practice than a few extra tiles of straight driving. This makes the planner strongly prefer the route with the fewest intersections, using tile count only as a tie-breaker between otherwise-equal routes. The heuristic used by A* is Manhattan distance to the goal, which never overestimates the true remaining cost, guaranteeing the search finds a truly shortest-cost route. The first move out of the start tile is additionally constrained to match the robot’s actual starting heading, so the planned route never begins with an immediate U-turn; if no route satisfies that constraint the search is retried without it.

Once a route is found, it is walked tile-by-tile to determine, for every intersection along the way, whether the robot must go straight, turn left, or turn right there – based on the heading it arrives with versus the heading it must leave with.

3.2 Executing Turns and Stop Lines

Left turns and straight crossings are executed as short pre-planned sequences of (speed, turn-rate, duration) steps, run open-loop for just long enough that normal lane-following can pick up the new lane afterward. Right turns instead use a closed-loop approach: the robot steers to keep the outer white line at a fixed target position in the camera image (“hugging”

the curve), which is more robust than a blind timed turn because it continuously reacts to what the camera actually sees. A stop line is detected as a wide, flat red rectangle near the bottom of the frame; on detection the robot halts completely for 2 seconds before proceeding, and stop-line detections are suppressed for 5 seconds afterward so the same line is not re-triggered while pulling away from it.

After the robot’s last planned turn, there are no further intersections to detect, so the system switches to “dead reckoning”: it estimates the remaining drive time from the number of tiles left (4.5s/tile) and stops once that time has elapsed. Time spent stopped for a red line or a duck along this final stretch is excluded from the countdown, so waiting does not falsely register as forward progress.

4 Object Detection and Avoidance

4.1 Duck Recognition

Ducks are yellow, the same color as the lane’s centre line, so they cannot be separated from lane markings by color alone. Instead, every yellow contour found in the lower part of the frame is passed through a shape classifier before being accepted as a duck:

- **Area** between 2,000 and 160,000 px – large enough to be a real nearby object, small enough not to be the lane line itself.
- **Aspect ratio** roughly duck-like (loosened for very large, close blobs, which are often clipped by the frame edge).
- **Solidity** and **rectangularity** together reject shapes that are essentially straight rectangles – i.e. lane markings – since a duck’s rounded silhouette is measurably less solid and less rectangular than a painted line segment.
- A polygon approximation (`approxPolyDP`) rejects any shape that simplifies to very few corner points *and* is highly solid, almost certainly a straight-edged object rather than a duck.

Any yellow pixels belonging to a confirmed duck are masked out of the yellow channel before it is used for lane detection, so a duck near the centre line cannot be mistaken for part of the lane.

4.2 Avoidance Behaviour

When a duck is detected close enough and large enough to matter (seen for 3 consecutive frames, to filter out one-frame glitches), the robot comes to a complete stop and watches it. If the duck’s position or apparent size keeps changing beyond a small threshold, the robot treats it as possibly walking away and keeps resetting a settle timer, up to a 2-second minimum and a 10-second hard cap regardless. If the duck clears the path on its own, the robot resumes lane following.

Otherwise, the robot commits to a go-around maneuver: for a fixed 3-second window, the same PD controller is re-pointed at the yellow centre line itself instead of the usual lane-center target, nudging the robot left and around the duck. Because

this maneuver is still closed-loop – reacting to the camera every frame, with a gentler steering limit for a smoother swerve – it cannot run away or overshoot the way a blind, fixed-duration turn could. Short cooldowns after each go-around and between separate duck episodes prevent the same duck from re-triggering the response as the robot drives past it.

5 Conclusion

The final system reliably drives duck3 from a fixed start tile to a fixed goal tile on the TUM track, following the lane, stopping correctly at red lines, planning and executing the necessary intersection turn via A*, and detecting and safely routing around ducks placed in its path. The entire pipeline is intentionally classical – HSV thresholding, a PD controller, and a graph-search planner – rather than learned, which made it straightforward to tune, debug, and explain from first principles, and it proved robust enough for repeated runs on the physical track.

Before settling on this approach, we also attempted a learning-based alternative: a PPO reinforcement-learning agent trained for 183 million simulation steps using stable-baselines3. The agent successfully learned to follow lanes inside the simulator, but exhibited persistent circular motion when deployed on the real robot. We attribute this to the sim-to-real gap – differences between the simulated and real camera images, dynamics, and lighting that the policy had not seen during training. We treat this as a useful negative result: it informed our decision to invest in a classical pipeline whose behaviour we could fully inspect and reason about, and which does not require a simulator that matches the real track and lighting closely enough to transfer.

We also implemented a standalone U-turn maneuver during development, but ultimately removed it: the planner’s fixed start and goal tiles never produce a route that requires one, so it added complexity without serving the actual driving task.

Given more time, natural extensions would include automatically re-tuning the HSV thresholds for a given lighting condition rather than fixing them by hand, generalizing the duck detector to different duck colors and sizes, and giving the robot a fallback behaviour for being blocked in a way the planned A* route did not anticipate.

Acknowledgement: The hardware and software platform used for these experiments was developed by the Duckietown Project – www.duckietown.org.

References

- [1] Duckietown Foundation, “Duckietown Platform,” www.duckietown.org.
- [2] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 1968.

- [4] Open Robotics, “ROS: Robot Operating System,” www.ros.org.
- [5] roslibpy documentation, “roslibpy: Python ROS Bridge library,” roslibpy.readthedocs.io.
- [6] A. Hill et al., “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, 2021.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.